PROGRAMAREA CALCULATOARELOR

FMI

Secția Calculatoare și tehnologia informației, anul I

Cursul 1 / 06 octombrie 2025

CUPRINSUL CURSULUI 1

0. Prezentarea cursului de

   Programarea calculatoarelor

1. Primul curs

camelia.obreja@unibuc.ro

UTILITATEA CURSULUI DE PC (PROGRAMAREA CALCULATOARELOR)

Programare procedurală:

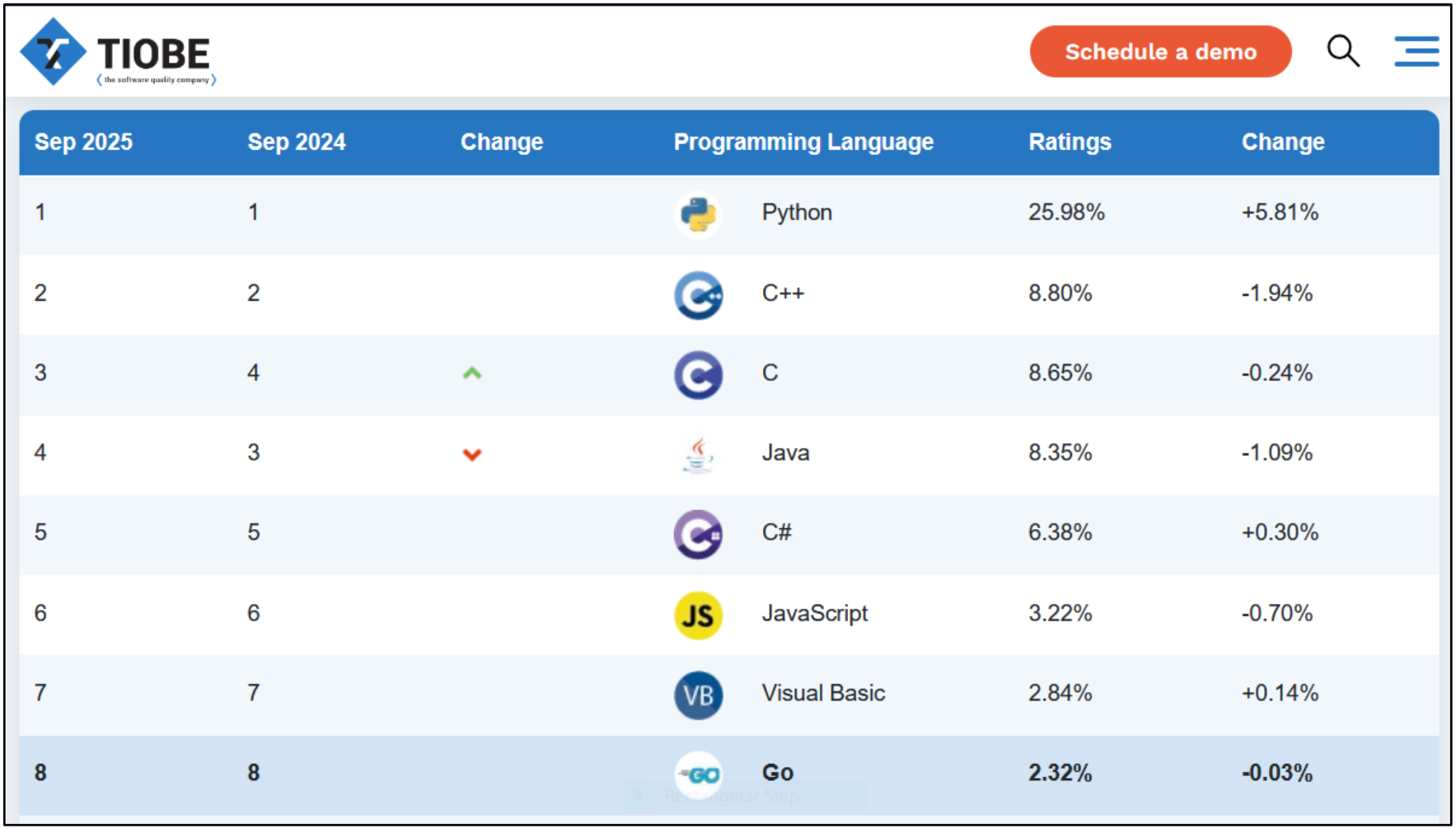
- paradigmă de programare bazată pe conceptul de apel de procedură/funcție/rutină/subrutină;
- un program este privit ca *o mulțime ierarhică de funcții* care manipulează datele.

Vom studia **limbajul C**:

- limbaj fundamental de programare (1970), exponent al programării procedurale;
- alte limbaje (*C++*, *Java*, *PHP*, *Python*, *etc.*) împrumută multe din caracteristicile limbajului C.

DE CE C?

http://www.tiobe.com/tiobe-index/



TIOBE
the software quality company

Schedule a demo

Sep 2025	Sep 2024	Change	Programming Language	Ratings	Change
1	1		Python	25.98%	+5.81%
2	2		C++	8.80%	-1.94%
3	4	▲	C	8.65%	-0.24%
4	3	▼	Java	8.35%	-1.09%
5	5		C#	6.38%	+0.30%
6	6		JavaScript	3.22%	-0.70%
7	7		Visual Basic	2.84%	+0.14%
8	**8**		**Go**	**2.32%**	**-0.03%**

MATERIALE

MSTeams. Cod echipă: f26mz1i



OBIECTIVELE CURSULUI

1. Formarea deprinderilor de programare structurată (modularizare) în limbaje de programare clasice si moderne (descompunerea unei probleme complexe în subprobleme relativ simple și independente)
2. Însușirea caracteristicilor limbajului C:
   - alocarea memoriei,
   - lucrul cu pointerii,
   - lucrul cu fișierele,
   - programarea generică.

   (*să codați în C, să vă dați seama ce face un cod scris de altcineva, să depanați cod în C*)
3. Dezvoltarea unei gândiri algoritmice + abilitate de programare (*foarte utile în rezolvarea diverselor probleme cu care vă veți întâlni în facultate sau în viața reală*)

PROGRAMA CURSULUI

1. Introducere

- Algoritmi
- Limbaje de programare.
- Introducere în limbajul C. Structura unui program C.

2. Fundamentele limbajului C

- Etapele realizării unui program C.
- Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii.
- Tipuri derivate de date: tablouri, șiruri de caractere, structuri, uniuni, câmpuri de biți, enumerări, pointeri
- Instrucțiuni de control
- Directive de preprocesare. Macrodefiniții.
- Funcții de citire/scriere.

3. Fișiere text

- Funcții specifice de manipulare.

4. Funcții (1)

- Declarare și definire. Apel. Metode de trasmitere a paramerilor. Pointeri la funcții.

5. Tablouri și pointeri

- Legătura dintre tablouri și pointeri
- Aritmetica pointerilor
- Alocarea dinamică a memoriei
- Clase de memorare

6. Șiruri de caractere

- Funcții specifice de manipulare

7. Fișiere binare

- Funcții specifice de manipulare

8. Structuri de date complexe și autoreferite

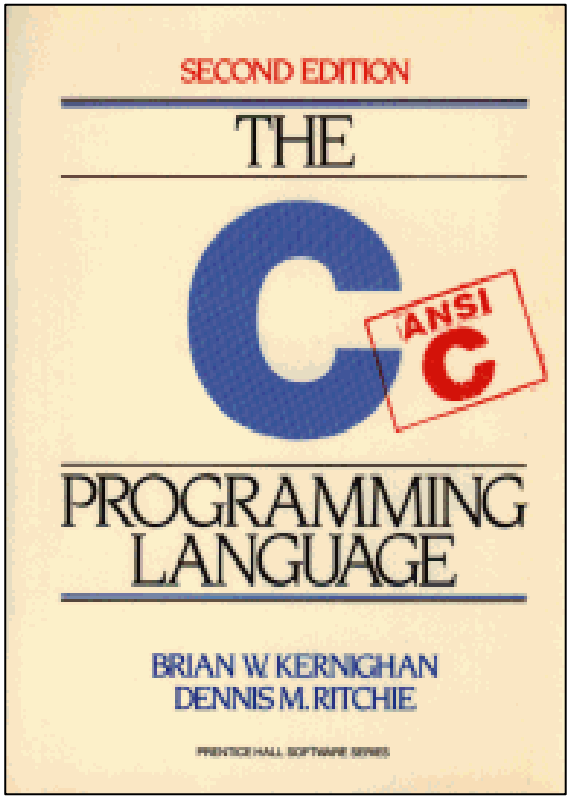
- Definire și utilizare

9. Funcții (2)

- Funcții cu număr variabil de argumente
- Preluarea argumentelor funcției main din linia de comandă
- Programare generică

BIBLIOGRAFIE

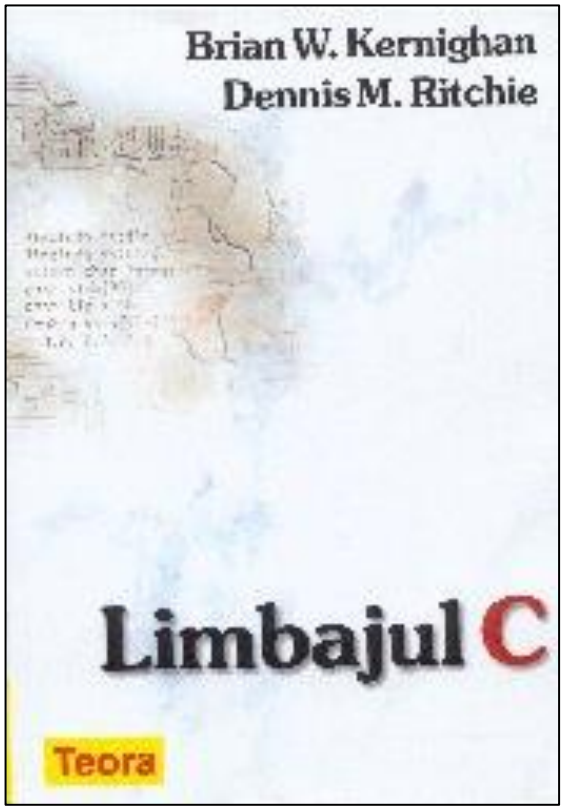
1. Kernighan & Ritchie: The C programming language
https://www.cs.sfu.ca/~ashriram/Courses/CS295/assets/books/C_Book_2nd.pdf



BIBLIOGRAFIE

1. Kernighan & Ritchie: The C programming language

2. Kernighan & Ritchie: Limbajul C, Editura Teora, 2003
https://www.slideshare.net/slideshow/manual-de-programare-c/15548927#67

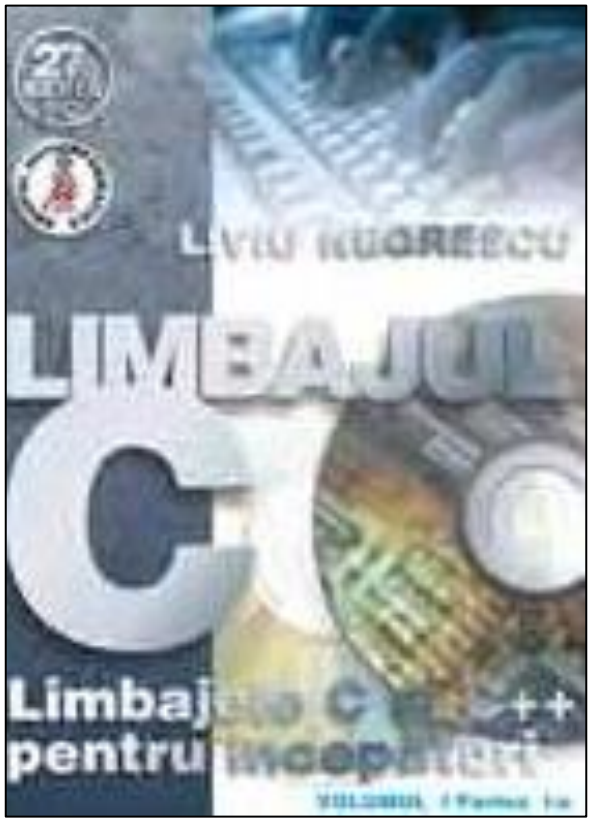


BIBLIOGRAFIE

1. Kernighan & Ritchie: The C programming language

2. Kernighan & Ritchie: Limbajul C, Editura Teora, 2003

3. Liviu Negrescu: Limbajele C si C++ pentru începători, volumul 1, partea I si II (Limbajul C), Editura Albastra, 2001
https://www.studocu.com/ro/document/universitatea-politehnica-din-bucuresti/baze-de-date/liviu-negrescu-limbajele-c-si-c-pentru-incepatori-partea-1-informatica-0-editur-albastra-libgen/43158615
sau
https://dokumen.pub/qdownload/limbajele-c-si-c-pentru-incepatori-partea-1-informatica.html



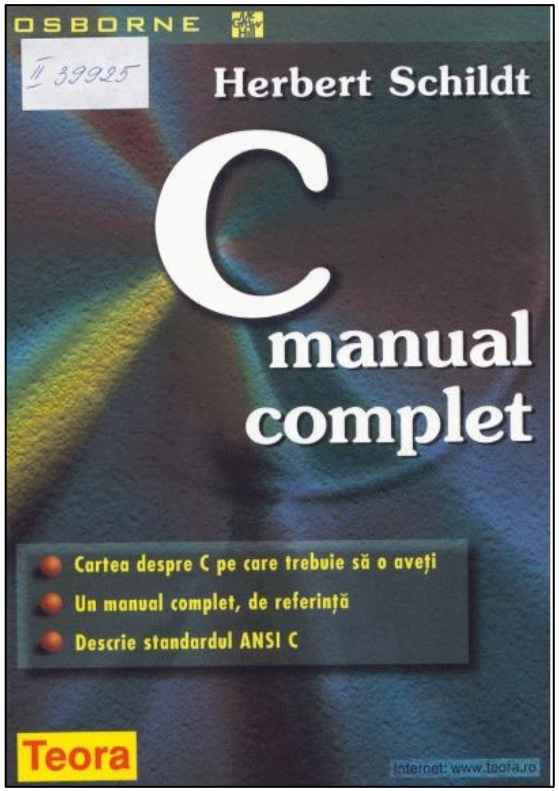
BIBLIOGRAFIE

1. Kernighan & Ritchie: The C programming language

2. Kernighan & Ritchie: Limbajul C, Editura Teora, 2003

3. Liviu Negrescu: Limbajele C si C++ pentru începători, volumul 1, partea I si II (Limbajul C), Editura Albastra, 2001

4. Herbert Schildt: C, manual complet. Editura Teora, 2002



BIBLIOGRAFIE

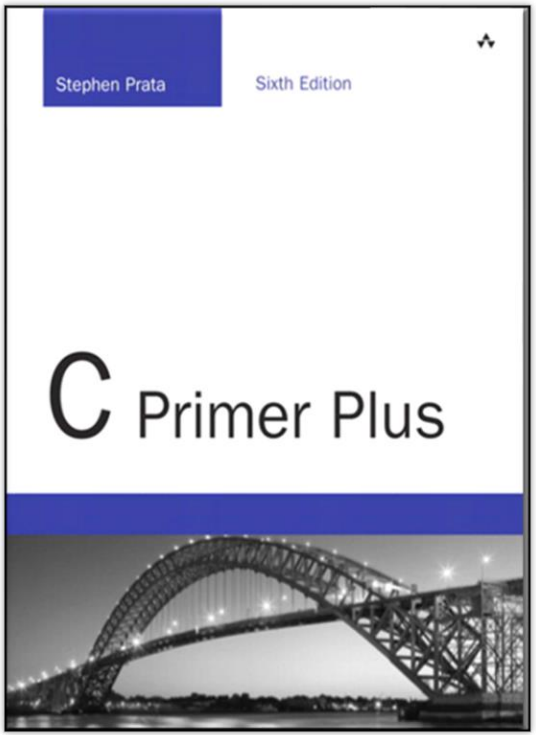
1. Kernighan & Ritchie: The C programming language

2. Kernighan & Ritchie: Limbajul C, Editura Teora, 2003

3. Liviu Negrescu: Limbajele C si C++ pentru începători, volumul 1, partea I si II (Limbajul C), Editura Albastra, 2001

4. Herbert Schildt: C, manual complet. Editura Teora, 2002

5. Stephan Prata: C primer plus, 6th Edition
https://raw.githubusercontent.com/hmsjy2017/free-ebooks/a82b972d3d26655599b3543903b3d80afed8dd90/C%20Primer%20Plus%20(6th%20Edition).pdf



BIBLIOGRAFIE

1. Kernighan & Ritchie: The C programming language

2. Kernighan & Ritchie: Limbajul C, Editura Teora, 2003

3. Liviu Negrescu: Limbajele C si C++ pentru începători, volumul 1, partea I si II (Limbajul C), Editura Albastra, 2001

4. Herbert Schildt: C, manual complet. Editura Teora, 2002

5. Stephan Prata: C primer plus, 6th Edition

6. S. Summit, Introductory C Programming Class Notes, http://www.eskimo.com/~scs/cclass/notes/top.html

REGULAMENT DE EVALUARE ȘI NOTARE

$$Nota = \min(10, Curs + Laborator + Seminar)$$

6p 4p 1p

Curs: evaluare finală prin probă scrisă (teorie, algoritmi și probleme): o notă cuprinsă între 1 și 10 (minim *5*)

Laborator: activitate în timpul orelor de laborator + teme pentru acasă: o notă cuprinsă între 1 și 10 (minim *5*)

Seminar: activitate în timpul orelor de seminar: o notă cuprinsă între 0 și 10

CUPRINSUL CURSULUI DE AZI

0. Prezentarea cursului de Programare Procedurală

1. Primul curs

CURSUL 1:

1. Algoritmi
2. Limbaje de programare
3. Introducere în limbajul C. Structura unui program C.

ALGORITMI

Rezolvarea oricărei probleme implică mai multe etape:

1. Analiza problemei
2. Găsirea soluției [optime]
3. Elaborarea algoritmului
4. Implementarea algoritmului într-un limbaj de programare
5. Verificarea corectitudinii algoritmului propus
6. Analiza complexității

ALGORITMI

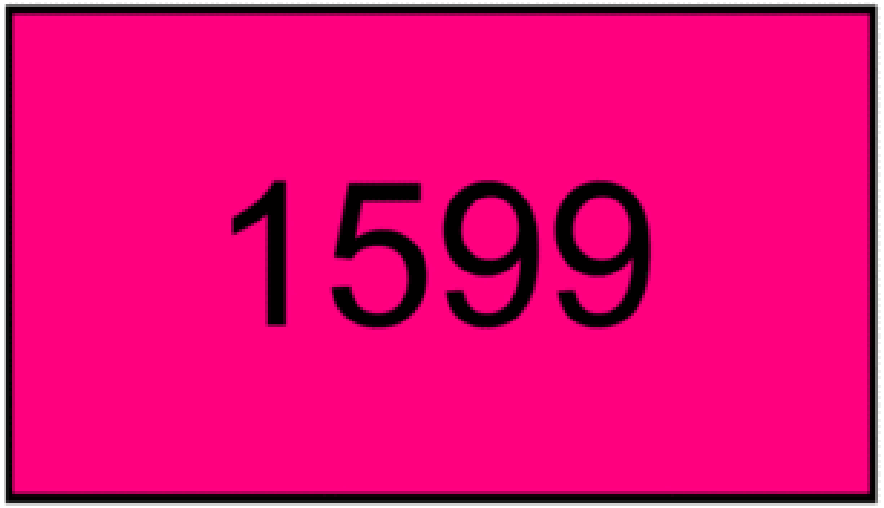
Algoritm = *o succesiune finită, ordonată și bine definită (exprimată clar și precis) de operații executabile (instrucțiuni, pași) care constituie o metodă corectă de rezolvare a unei probleme pornind dintr-o stare inițială, folosind datele disponibile și ajungând în starea finală dorită*.

ALGORITMI

Exemplu: algoritmul lui Euclid (cu împărțiri) pentru determinarea celui mai mare divizor comun a două numere naturale

cmmdc(1599, 650) = ?

```
1599 = 650×2 + 299
650 = 299×2 + 52
299 = 52×5 + 39
52 = 39×1 + 13
39 = 13×3 + 0
```



cmmdc(1599, 650) = 13

https://en.wikipedia.org/wiki/Algorithm

ALGORITMI

Exemplu: algoritmul lui Euclid (cu împărțiri) pentru determinarea celui mai mare divizor comun a două numere naturale

main.cpp

```
1  #include <iostream>
2  using namespace std;
3  int main()
4  {
5      int n, m;
6      cin >> n >> m;
7      while(m != 0)
8      {
9          int r = n % m;
10         n = m;
11         m = r;
12     }
13     cout << n << endl;
14     return 0;
15 }
```

REPREZENTAREA ALGORITMILOR

1. Pseudocod/ limbaj natural
2. Schemă logică
3. Program într-un limbaj de programare

1. PSEUDOCOD

- ✓ limbaj natural structurat exprimat formal
- ✓ fiecare pas al algoritmului este reprezentat de o linie separată, ca o propoziție
- ✓ acțiuni (verbe) aplicate unor date (substantive)
- ✓ indentarea poate reda ierarhia instrucțiunilor

1. PSEUDOCOD

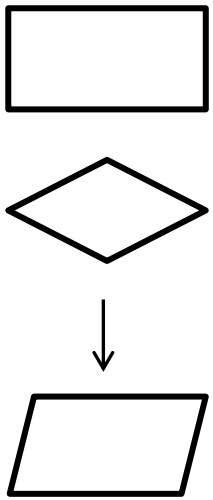
- ✓ fiecare pas al algoritmului este reprezentat de o linie separată, ca o propoziție
- ✓ indentarea poate reda ierarhia instrucțiunilor

Algoritmul lui Euclid
(scăderi repetate)

```
cât timp B <> A execută
    dacă A > B atunci
        A = A – B
    altfel
        B = B – A
afișează A
```

2. SCHEMĂ LOGICĂ

- alăturare de simboluri vizuale care desemnează fluxul logic al pașilor



bloc de instrucțiuni

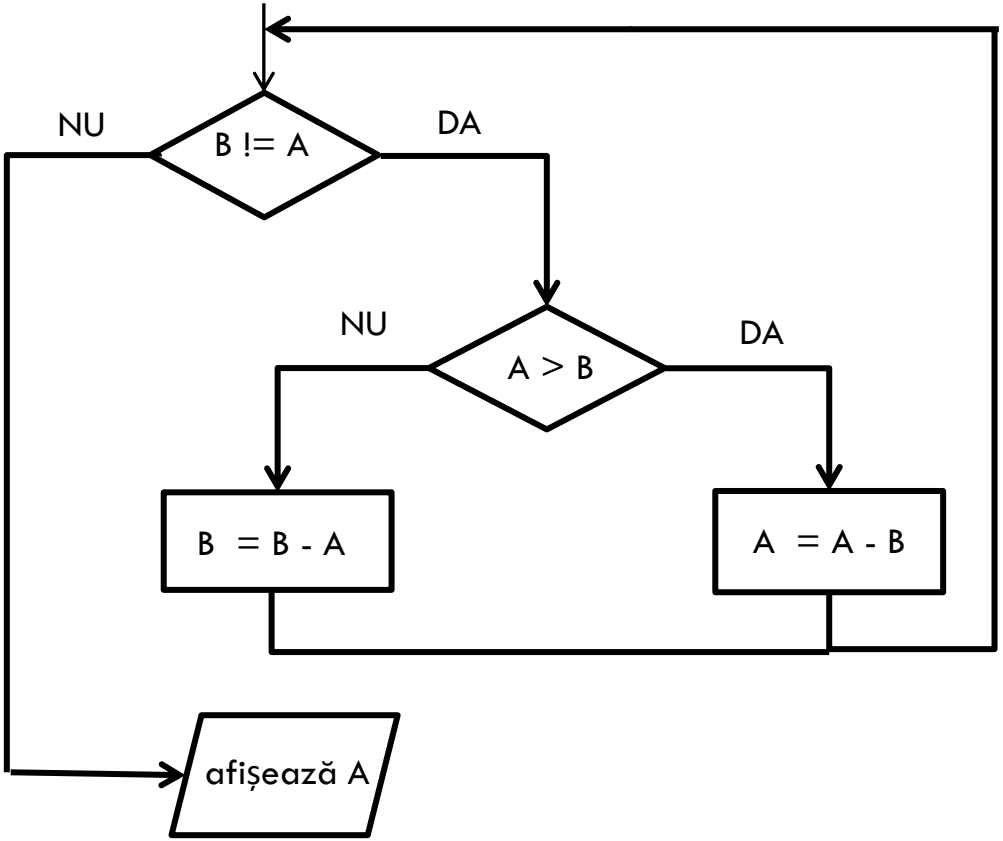
structura alternativă

direcția fluxului

operația de intrare/ieșire

2. SCHEMĂ LOGICĂ

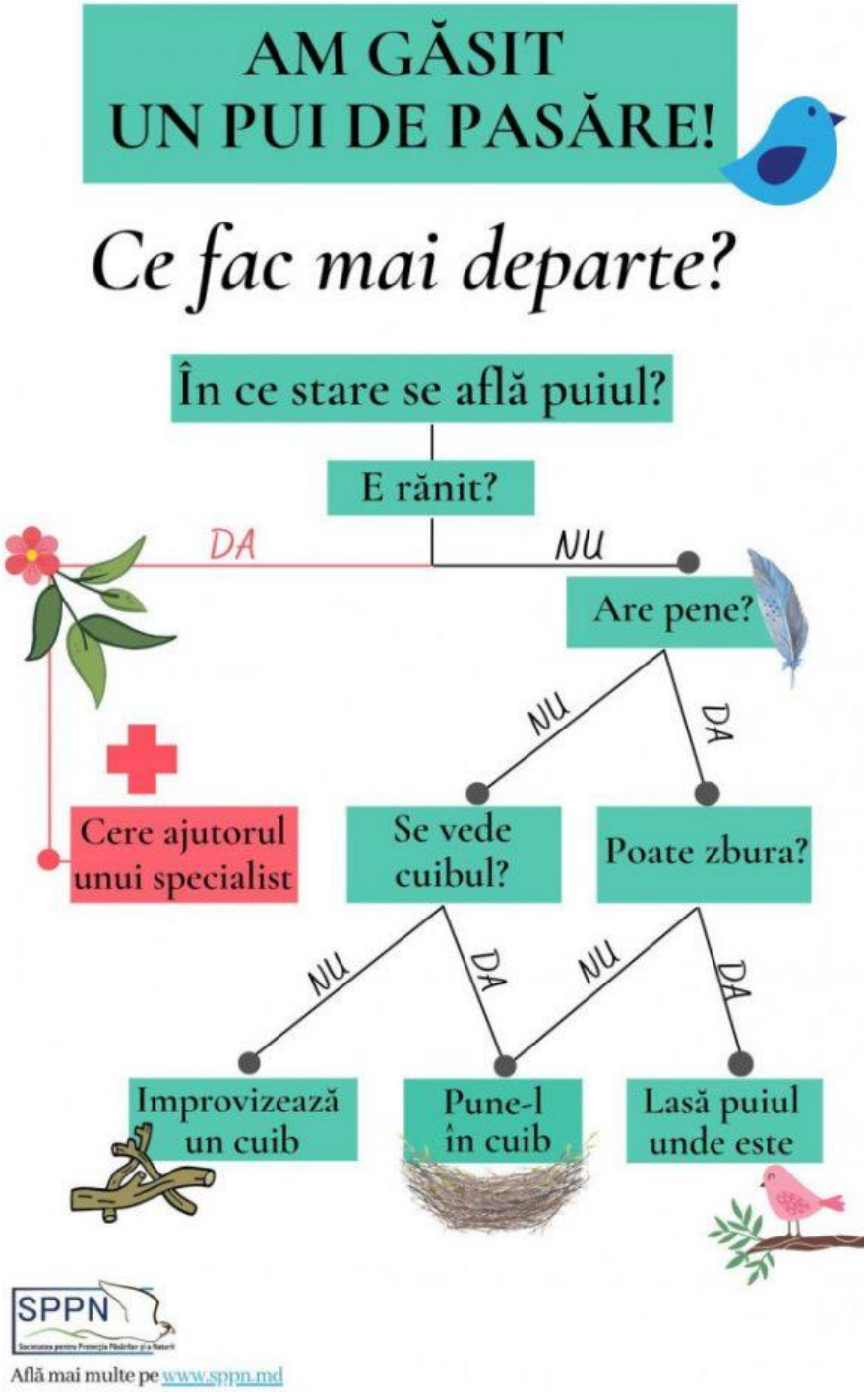
-alăturare de simboluri vizuale care desemnează fluxul logic al pașilor



Algoritmul lui Euclid
(scăderi repetate)

```
cât timp B != A
  dacă A > B
     A = A – B;
  altfel
     B = B – A;
afișează A
```

2. SCHEMĂ LOGICĂ



3. PROGRAM

- set de instrucțiuni într-un limbaj de programare

main.cpp

```cpp
#include <iostream>
using namespace std;
int main()
{
    int A, B;
    cin >> A >> B;
    while(A != B)
    {
        if (A > B)
            A = A - B;
        else
            B = B - A;
    }
    cout << A << endl;
    return 0;
}
```

CURSUL 1:

1. Algoritmi
2. Limbaje de programare
3. Introducere în limbajul C. Structura unui program C.

LIMBAJE DE PROGRAMARE

Rezolvarea oricărei probleme implică mai multe etape:

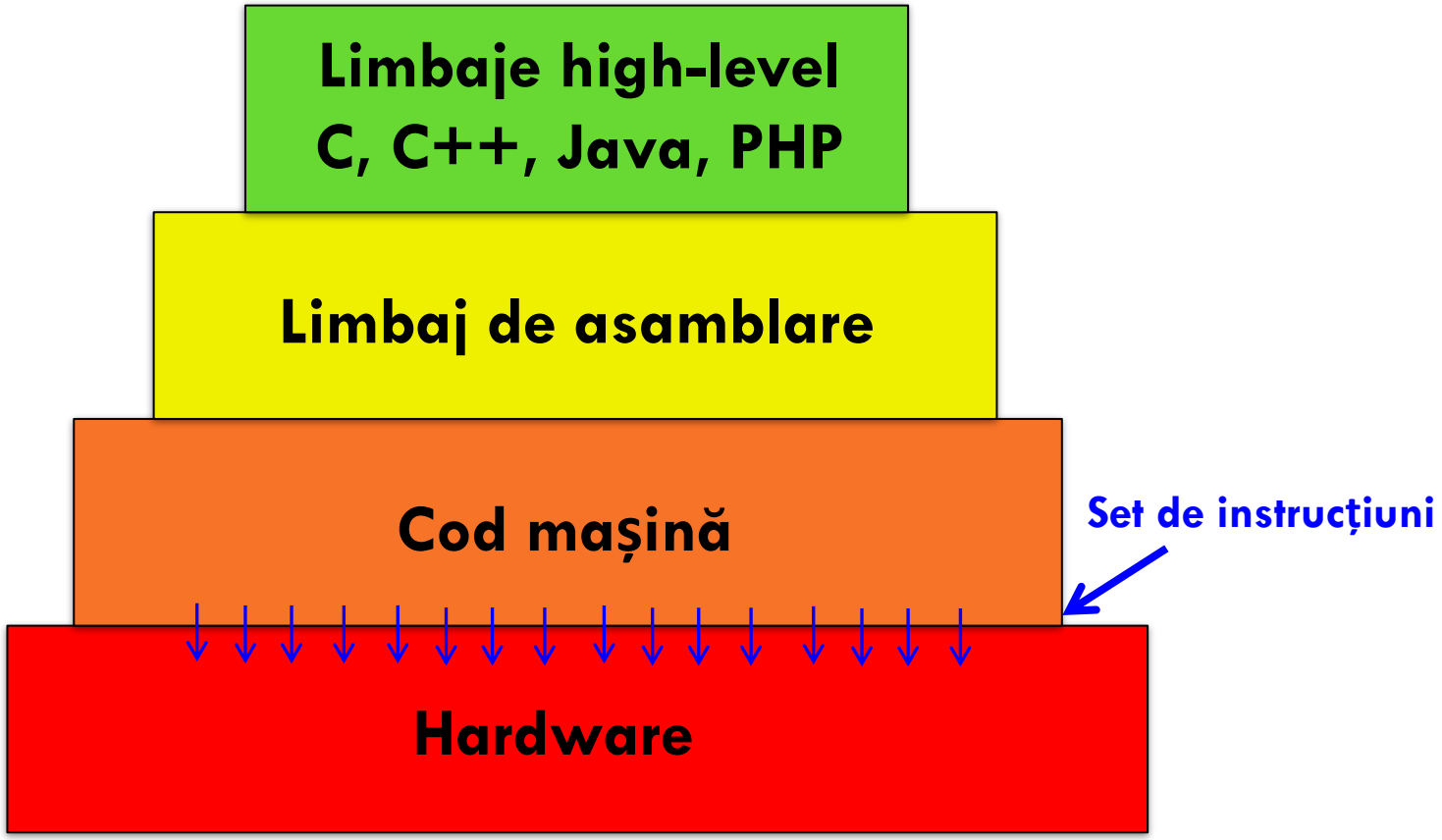
1. Analiza problemei
2. Găsirea soluției [optime]
3. Elaborarea algoritmului
4. Implementarea algoritmului într-un limbaj de programare
5. Verificarea corectitudinii algoritmului propus
6. Analiza complexității

LIMBAJ DE PROGRAMARE

- limbaj artificial cu sintaxă și semantică bine definite
- pune la dispoziția programatorilor construcții sintactice prin care sunt specificate succesiunea de operații/ instrucțiuni elementare (pe care un calculator le poate executa) asociate algoritmului de rezolvare a unei probleme
- este necesară cunoașterea setului de operații/instrucțiuni elementare al calculatorului la care ne referim
- limbaj mașină = limbajul nativ al unui calculator (mașină)

LIMBAJE LOW-LEVEL ȘI HIGH-LEVEL

- date de apropierea unui limbaj de limbajul nativ al calculatorului (limbaj mașină = cod mașină)



LIMBAJE LOW-LEVEL (DE NIVEL SCĂZUT)

Limbaj mașină

- limbajul nativ al unui calculator (mașină);
- șabloane de numere binare (reprezintă modul binar de codificare a instrucțiunilor și datelor în memorie)
- depinde de arhitectura sistemului



Instrucțiuni în limbaj mașină

Limbaj de asamblare

- în loc de cod mașină folosește o desemnare simbolică a elementelor programului (instrucțiuni, date)
- 01011011 = ADD, 01011100 = SUB

```
swap:
        muli $2, $5,4
        add  $2, $4,$2
        lw   $15, 0($2)
        lw   $16, 4($2)
        sw   $16, 0($2)
        sw   $15, 4($2)
        jr   $31
```

Instrucțiuni în limbaj de asamblare

LIMBAJE HIGH-LEVEL (DE NIVEL ÎNALT)

- cuprind mecanisme de exprimare apropiate de limbajul natural;
- folosesc:
  - **verbe** pentru a desemna acțiuni (**do, repeat, read, write, continue, switch, call, goto**, etc.),
  - **conjunctii** (**if, while**),
  - **adverbe** (**then, else**),
  - mecanisme de declarare și definire.

```
swap(int v[], int k)
{int temp;
   temp = v[k];
   v[k] = v[k+1];
   v[k+1] = temp;
}
```

Instrucțiuni în limbajul C

```
swap:
        muli $2, $5,4
        add  $2, $4,$2
        lw   $15, 0($2)
        lw   $16, 4($2)
        sw   $16, 0($2)
        sw   $15, 4($2)
        jr   $31
```

Instrucțiuni în limbaj de asamblare

```
00000000101000010000000000011000
00000000000011000000110000100001
10001100011000100000000000000000
10001100111100100000000000000100
10101100111100100000000000000000
10101100011000100000000000000100
00000011111000000000000000001000
```

Instrucțiuni în limbaj mașină

Reprezentările în limbajul C, limbaj de asamblare și limbaj mașină a instrucțiunilor de interschimbare a două valori

LIMBAJE HIGH-LEVEL (DE NIVEL ÎNALT)

- au o **descriere sintactică și semantică** bine definită
- descurajează greșelile de programare
- sunt **independente de procesor** (pentru asigurarea portabilității codului)
- sunt **independente de sistemul de operare** (pentru a permite realizarea de software multi-platforma)
- codul sursă se convertește în cod mașină folosind **compilatoare** sau **interpretoare**

CURSUL 1:

1. Algoritmi
2. Limbaje de programare
3. Introducere în limbajul C. Structura unui program C.

LIMBAJUL C

- popular, rapid și independent de platformă
- limbaj utilizat cel mai adesea pentru scrierea programelor eficiente și portabile: sisteme de operare, aplicații embedded, compilatoare, interpretoare, etc.
- dezvoltat la începutul anilor 1970 în cadrul Bell Laboratories de către Dennis Ritchie
  - strâns legat de sistemele de operare UNIX
- stă la baza pentru majoritatea limbajelor ”moderne”: C++, Java, C#, Javascript, Objective-C, etc.

STANDARDE OFICIALE ACTIVE ALE LIMBAJULUI C

C89 (C90)

- aprobat în 1989 de ANSI (American National Standards Institute) și în 1990 de către ISO (International Organization for Standardization)
- a eliminat multe din incertitudinile legate de sintaxa și gramatica limbajului
- cele mai multe compilatoare de C sunt compatibile cu acest standard (ANSI C)

C99

- aprobat în 1999, include corecturile aduse C89 dar și o serie de caracteristici proprii care în unele compilatoare apăreau ca extensii ale C89 până atunci
- compilatoarele oferă suport limitat și în multe cazuri incomplet pentru acest standard

C11

- aprobat în 2011; rezolvă erorile apărute în standardul C99 și introduce noi elemente, însă suportul pentru C11 este și mai limitat decât suportul pentru C99, majoritatea compilatoarelor nu s-au adaptat încă la acest standard

CARACTERISTICI ALE LIMBAJULUI C

- ***Limbaj procedural, structurat, de nivel de mijloc, scurt:***
- limbaj procedural, structurat
  - instrucțiuni specificate sub forma unor comenzi grupate într-o ierarhie de subprograme (denumite funcții) și care pot forma module
- limbaj compilat
  - compilatorul transformă instrucțiunile limbajului C în limbaj mașină
- limbaj de nivel de mijloc
  - permite accesul la date și comenzi aflate aproape de nivelul fizic folosind o sintaxă specifică limbajelor de nivel înalt
- limbaj scurt
  - număr redus de cuvinte cheie
  - multe funcționalități nu sunt incluse în limbajul de bază ci necesită includerea unor biblioteci standard

CARACTERISTICI ALE LIMBAJULUI C

- ***Limbaj eficient, portabil, permisiv**, poate fi **dificil de înțeles:***

- limbaj eficient
  - viteză mare de execuție a programelor, destinat și aplicațiilor implementate în limbaj de asamblare
  - reutilizarea ulterioară a subprogramelor
- limbaj portabil
  - limbaj independent de hardware
- limbaj permisiv
  - impune puține constrângeri, dă credit programatorului
  - permite introducerea unor erori care sunt foarte greu de depistat
- limbaj dificil de înțeles
  - un stil de programare adecvat este foarte important
  - obfuscated C code contest: www.ioccc.org

CUVINTE CHEIE

C89 = ANSI C : 32 de cuvinte cheie

auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

C99: ANSI C + alte 5 cuvinte cheie

```
_Bool  _Complex  _Imaginary  inline  restrict
```

STRUCTURA GENERALĂ A UNUI PROGRAM C

- modulul principal (funcția main)
- zero, unul sau mai multe module (funcții/proceduri) care comunică între ele și/sau cu modulul principal prin intermediul parametrilor și/sau a unor variabile globale

✓ unitatea de program cea mai mică și care conține cod este funcția/procedura și conține:

- partea de declarații/definiții
- partea imperativă (comenzile care se vor executa)

PRIMUL PROGRAM C

main.c

```c
#include <stdio.h>

int main()
{
    printf("Primul program scris in C.");
    return 0;
}
```

PRIMUL PROGRAM C

main.c

```c
#include <stdio.h>

int main()
{
    printf("Primul program scris in C.");
    return 0;
}
```

main.cpp

```cpp
#include <iostream>

int main()
{
    std::cout<<"Hello World!";
    return 0;
}
```

Algoritmi echivalenți

PRIMUL PROGRAM C

main.c

```c
#include <stdio.h>

int main()
{
    printf("Primul program scris in C.");
    return 0;
}
```

Algoritmi echivalenți

main.cpp

```cpp
#include <iostream>
using namespace std;

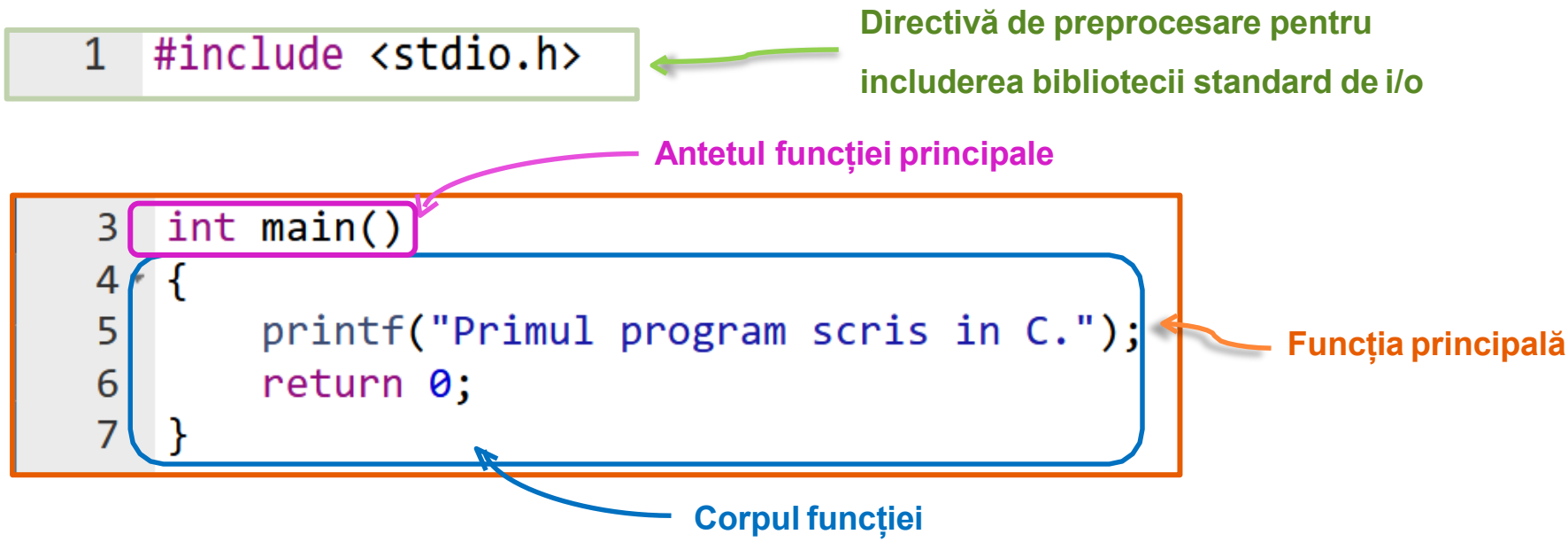
int main()
{
    cout<<"Hello World!";
    return 0;
}
```

main.cpp

```cpp
#include <iostream>

int main()
{
    std::cout<<"Hello World!";
    return 0;
}
```

PRIMUL PROGRAM C EXPLICAT



PRIMUL PROGRAM C EXPLICAT

```c
#include <stdio.h>

int main()
{
    printf("Primul program scris in C.");
    return 0;
}
```

Observații:

- main **nu** este cuvânt cheie în limbajul C, îl utilizăm pentru numirea funcției principale,
- printf **nu** este cuvânt cheie, este funcție de bibliotecă: *print (afișare) + f (format)*,
- limbajul C este case sensitive (face diferență între litere mici și mari),
- toate cuvintele cheie se scriu cu litere mici,
- instrucțiunile se termină cu caracterul ; (punct și virgulă),
- mai multe instrucțiuni pot fi scrise pe aceeași linie,
- spațiile ajută la organizarea codului.

STRUCTURA UNUI PROGRAM C SIMPLU

directive de preprocesare

```
int main()
{
        instrucţiuni
}
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("Primul program scris in C.");
6      return 0;
7  }
```

- Directive de preprocesare:
  - directive de definiție: #define N 10
  - directive de includere a bibliotecilor: #include <stdio.h>
  - directive de compilare condiționată: #if, #ifdef, ...
  - alte directive (vorbim în cursurile următoare)

- Funcții:
  - grupări de instrucțiuni sub un nume,
  - returnează o valoare sau se rezumă la efectul produs,
  - funcții scrise de programator vs. funcții furnizate de biblioteci,
  - programul poate conține mai multe funcții,
    - main este obligatoriu,
  - antetul și corpul funcției.

STRUCTURA UNUI PROGRAM C SIMPLU

directive de preprocesare
```
int main()
{
        instrucțiuni
}
```

```c
#include <stdio.h>

int main()
{
    printf("Primul program scris in C.");
    return 0;
}
```

- **Instrucțiuni:**
  - formează corpul funcțiilor
  - sunt exprimate sub formă de comenzi
  - **5 tipuri de instructiuni**:
    - ✓ instrucțiunea declarație
    - ✓ instrucțiunea atribuire
    - ✓ instrucțiunea apel de funcție
    - ✓ instrucțiuni de control
    - ✓ instrucțiunea vidă
  - toate instrucțiunile (cu excepția celor compuse) se termină cu caracterul ”;”
    - ✓ caracterul ; nu are doar rol de separator de instrucțiuni, ci instrucțiunile încorporează caracterul ; ca ultim caracter
    - ✓ omiterea caracterului ; reprezintă eroare de sintaxă

EXEMPLU DE PROGRAM C SIMPLU

- **citirea datelor:** funcția **scanf:** scanf("%format", &variabila);

Exemplu:

```
int x, y; double z;
scanf("%d", &x);
scanf("%d%lf", &y, &z);
```

- **afișarea datelor:** funcția **printf:** printf("%format", expresie);

Exemplu:

```
int x, y; scanf("%d %d", &x, &y); printf("%d\n", x+2);
printf("x este egal cu %d, iar y =%d", x, y);
```

- Lista completă a formatelor pentru citire și afișare cu funcții din biblioteca ***stdio.h*** urmează să fie studiate în cursul 2, odată cu tipurile de date din limbajul C.

(1) EXEMPLU DE PROGRAM C SIMPLU

```c
#include <stdio.h>

int main()
{
    int a, b;
    printf("dati doua numere intregi: ");
    scanf("%d %d", &a, &b);
    printf("cele doua numere sunt x = %d si y = %d\n", a, b);
    while (b != a)
    {
        if(a>b)
            a=a-b;
        else
            b=b-a;
    }
    printf("cmmdc = %d", b);
    return 0;
}
```

```c
#include <stdio.h>

int main()
{
    int a, b;
    printf("dati doua numere intregi: ");
    scanf("%d %d", &a, &b);
    printf("cele doua numere sunt x = %d si y = %d\n", a, b);
    while (b != a)
    {
        if(a>b)
            a=a-b;
        else
            b=b-a;
    }
    printf("cmmdc = %d", b);
    return 0;
}
```

Algoritmi echivalenți

```cpp
#include <iostream>
using namespace std;
int main()
{
    int a, b;
    cout<<"dati doua numere intregi: ";
    cin>>a>>b;
    cout<<"cele doua numere sunt x = "<<a<<" si y = "<<b<<"\n";
    while (b != a)
    {
        if(a>b)
            a=a-b;
        else
            b=b-a;
    }
    cout<<"cmmdc = "<<b;
    return 0;
}
```

(2) EXEMPLU DE PROGRAM C SIMPLU

```c
#include <stdio.h>

int main()
{
    int n, a, max;
    scanf("%d", &n);
    scanf("%d", &a);
    n=n-1;
    max=a;
    while (n)
    {
        scanf("%d", &a);
        if (a>max)
            max=a;
        n=n-1;
    }
    printf("max = %d", max);
    return 0;
}
```

(3) EXEMPLU DE PROGRAM C SIMPLU

```c
#include <stdio.h>
int main()
{
    int n, p, u, ok;
    scanf("%d", &n);
    scanf("%d", &p);
    n = n - 1;
    ok = 1;
    while (n > 0)
    {
        scanf("%d", &u);
        if (u < p)
            ok = 0;
        n=n-1;
        p=u;
    }
    if (ok == 1)
        printf("DA");
    else
        printf("NU");
    return 0;
}
```

STRUCTURA UNUI PROGRAM C COMPLEX

```
comentarii
directive de preprocesare
declarații și definiții globale

int main()
{
        declarații și definiții locale
        instrucțiuni
}
```

STRUCTURA UNUI PROGRAM C

Comentariile:

- formă de documentare a codului sursă, sunt ignorate de compilator

- **2 tipuri de comentariu**:
  - încep cu /* și se termină cu */:
    - se pot extinde pe mai multe linii,
    - nu se pot imbrica,
    - utile pentru inserarea unor explicații mai lungi

  - încep cu // și se termină la sfârșitul liniei:
    - utile pentu comentariile inserate pe marginea codului (apare în C99, nu este în C89)

PROGRAMAREA CALCULATOARELOR

Curs 1:

0. Descrierea cursului: Obiective, Conținuturi/ Programa, Bibliografie, Notare
1. Algoritmi. Metode de reprezentare
2. Limbaje de programare
3. Limbajul C:
    a) Introducere
    b) Caracteristici ale limbajului C
    c) Structura unui program C

Curs 2: Fundamentele limbajului C:

1. Etapele realizării unui program C.
2. Tipuri de date fundamentale. Variabile. Constante. Operatori. Expresii. Conversii.